

LT10d : ADT Hash Table

Part 2 : About Hash Table (without Collision Resolution)

Basic steps for storing data into a Hash Table:

1. Write the Hash Function
2. Initialize an empty Hash Table
3. Calculate the Hash Value, i , of the data
4. Store the data in the Hash Table at index i if it is empty

(1) A Simple Hash Function:

```
def hash(string):  
    total = 0  
    for i in range(len(string)):  
        total += ord(string[i]) * (i+1)  
    return total
```

```
>>> hash('any_string') % n
```

This **n** value depends on the size of the hash table or array, so we will take this out of the hash function.

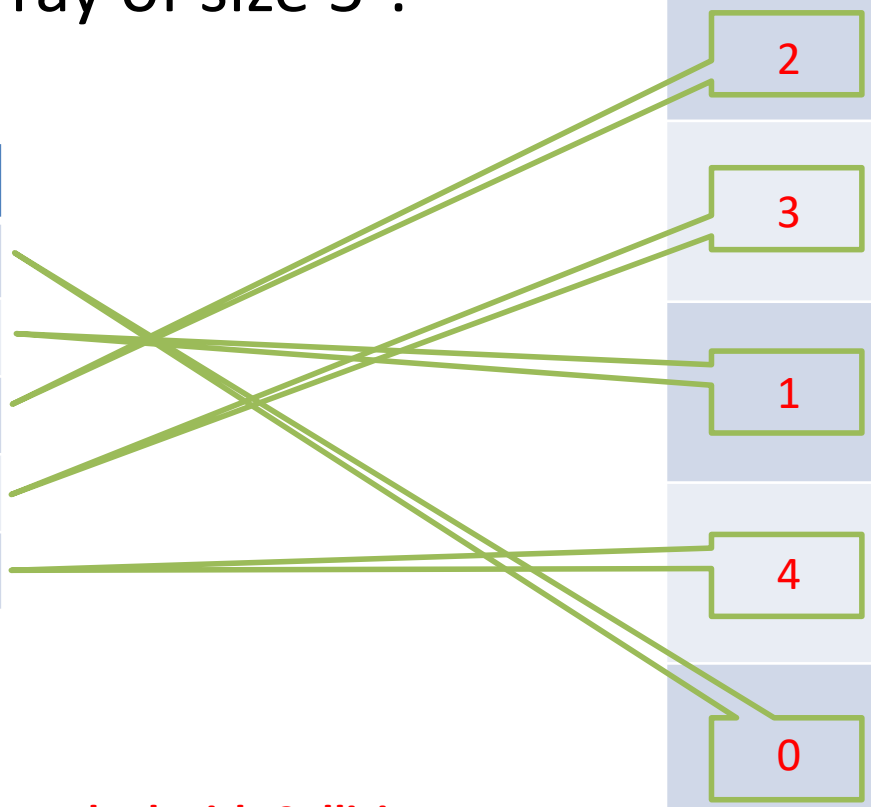
Store data in a Hash Table / Array:

```
>>> data = ['cdab', 'dbac', 'adcb', 'acbd', 'abcd']
```

How to store the data into an array of size 5 ?

Array Index	
0	'abcd'
1	'adcb'
2	'cdab'
3	'dbac'
4	'acbd'

Hash value % 5	Hash value	string
2	982	'cdab'
3	983	'dbac'
1	986	'adcb'
4	989	'acbd'
0	990	'abcd'



We will ignore all other data strings for now and not deal with Collision yet.

(2) Initialize an empty Hash Table:

```
def init_table(n):  
    table = []      # declare  
    table += [''] * n # initialize  
    return table
```

(3) Store the data into the Hash Table:

```
def hashtable(seq):  
    tbl = init_table(len(seq))  
    for ele in seq:  
        i = hash(ele) % len(seq)  
        if tbl[i] == '':  
            tbl[i] = ele  
        else:  
            #No Collision Resolution  
            print(ele, ' is not added.')  
    return tbl
```



We will not
handle collision
at this moment.

```
>>> data = ['cdab', 'dbac', 'adcb', 'acbd', 'abcd']  
>>> data_table = hashtable(data)
```

data_table	
Array Index	
0	'abcd'
1	'adcb'
2	'cdab'
3	'dbac'
4	'acbd'

(3) Store the data into the Hash Table:

```
def hash
```

Why do we want
to store the data
in a Hash Table???

```
return
```

```
' is not added.')
```

```
return tbl
```

```
>>> data = ['cdab', 'dbac', 'adcb', 'acbd', 'abcd']
```

```
>>> data_table = hashtable(data)
```

data_table	
Array Index	
0	'abcd'
1	'adcb'
2	'cdab'
3	'dbac'
4	'acbd'

Query

Without doing a **linear search**, ie checking every element, can we know whether '**cdab**' or '**bdca**' is in the table?

```
def hash(string):
    total = 0
    for i in range(len(string)):
        total += ord(string[i])*(i+1)
    return total
```

```
>>> hash('cdab') % 5
```

```
>>> 2
```

```
>>> hash('bdca') % 5
```

```
>>> 3
```

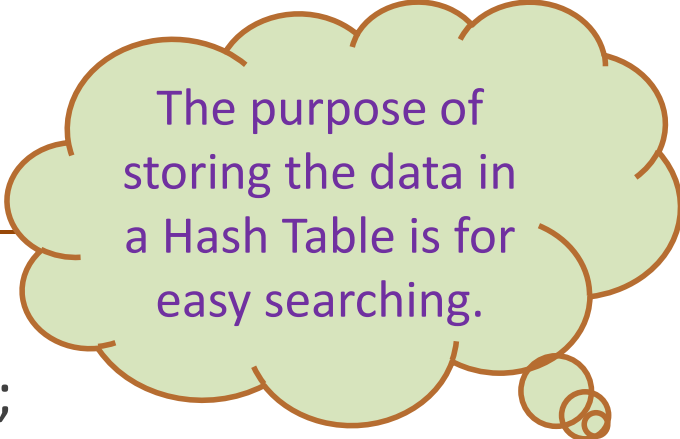
Yes, '**cdab**' is in index **2**, so it is in the table.

data_table	
Array Index	
0	'abcd'
1	'adcb'
2	'cdab'
3	'dbac'
4	'acbd'

But '**bdca**' is not in index **3**, so he is not in the array.

Steps for checking if an item exist in the Hash Table:

1. Calculate the Hash Value, i , of the item;
2. The Hash Table at index i may be empty or contained a data;
 - return True if the data is the same as the item, else return False.



The purpose of storing the data in a Hash Table is for easy searching.

Searching if an item exists in the Hash Table:

```
>>> data = ['cdab', 'dbac', 'adcb', 'acbd', 'abcd']
```

```
>>> data_table = hashtable(data)
```

```
def search(table, item):
```

Write the Python code for searching an item
in a Hash Table (without Collision Resolution).

```
>>> search(data_table, 'cdab')
```

```
>>> search(data_table, 'bdca')
```

data_table	
Array Index	
0	'abcd'
1	'adcb'
2	'cdab'
3	'dbac'
4	'acbd'